Prediction Models

Communication Modeling

In parallel and distributed systems:

- Processes communicate by exchanging messages
- Communication time significantly affects overall performance

Communication cost depends on:

- Network latency
- Available bandwidth
- Processor involvement
- Message size

Observation

- Small messages behave differently from large messages
- For large messages, data is transferred **gradually**, not instantly

Need

→ A simple analytical model that can estimate communication time based on system characteristics

LogP Family to LogGOPS

LogP

- Designed for small messages
- Parameters:
  - **L** – network latency
  - **o** – CPU overhead per message
  - **g** – minimum time between message injections
  - **P** – number of processes
- Assumes contention-free network and fixed-size messages

LogGP

- Extends LogP for large messages
- Adds:
  - **G** – transmission time per byte
- Captures bandwidth effects

LogGPS

- Models synchronization behavior in large-message protocols

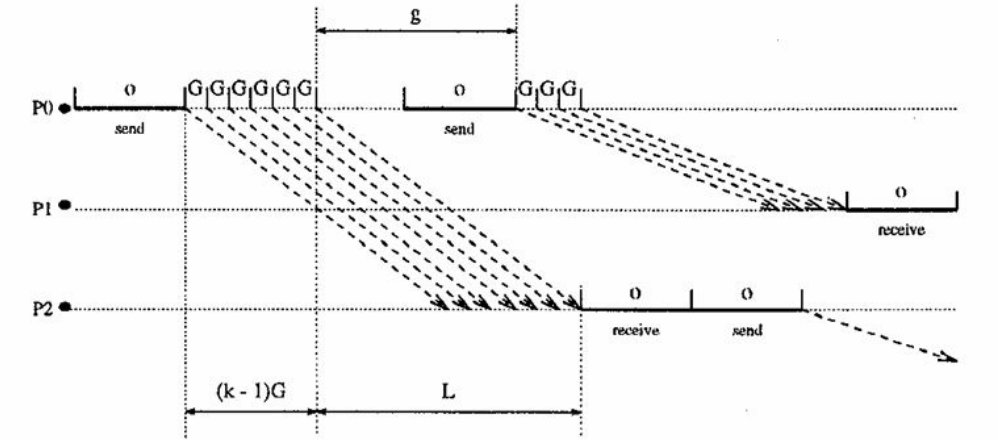
  Adds:
  - **S** – message size threshold for rendezvous (sender waits for receiver)

LogGOPS

- Further refinement for modern systems
- Adds:
  - **O** – CPU overhead per byte
- Enables accurate modeling of:
  - Large messages
  - Communication–computation overlap
  - Modern network behavior

Measuring LogGOPS Parameters

- Sender first spends **o** time to initiate the message (CPU overhead).
- For a message of size **k bytes**:
  - Network transmission cost = **(k − 1)G**
  - CPU processing per byte = **(k − 1)O**
- The message then experiences **network latency L**.
- At the receiver, CPU spends **o** time to process the message.
- **Final LogGOPS Communication Time** $T = 2o + L + (k-1)(G+O)$



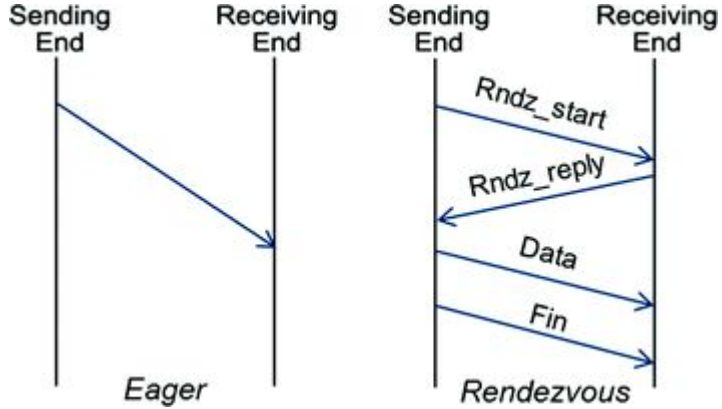
L – Network latency (time for a message to reach destination)
o – Communication overhead per message (CPU involvement at sender/receiver)
g – Gap between consecutive messages (injection rate limit)
G – Gap per byte (transmission time per byte → inverse bandwidth)
P – Number of processes
O - Per-message overhead gap

Hoefler, Torsten, Timo Schneider, and Andrew Lumsdaine. "LogGOPSim: simulating large-scale applications in the LogGOPS model." *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing*. 2010.
Hoefler, Torsten, et al. "Netgauge: A network performance measurement framework." *International Conference on High Performance Computing and Communications*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007.

What is S in LogGOPS?

S — Synchronization Threshold (Rendezvous Threshold)

- **S** is the **message size threshold** that determines when sender–receiver synchronization is required.
  If message size < S
  → **Eager protocol**
  → **Sender sends data immediately (no waiting).**
  If message size ≥ S
  → **Rendezvous protocol**
  → **Sender must wait until the receiver is ready (buffer availability).**



- This introduces **additional synchronization delay** in communication.
- Conceptually:
  - **S separates eager vs rendezvous behavior**
  - Helps model **real MPI behavior** for large messages.

Compute Factor C

Why do we need C?

- Communication models (LogP/LogGP/LogGOPS) consider:
  - Network delay
  - CPU overhead for sending/receiving

- **But collective operations also perform local computation**
  - e.g., reduction (sum, max, min)
  - memory copy and buffer processing

Total local computation cost => $T = k \cdot C$

- **k** = message size (bytes or elements)
- **C** = compute + memory time per byte
- Larger messages → **more local processing**

Limitations of LogGOP-Based Models

- Considers only network-level parameters (latency, bandwidth, overhead); does not capture full system behavior.
- Does not model internal implementations of MPI collectives or their runtime behavior for real applications.
- Assumes a simple communication model; does not capture network contention, topology effects, or congestion in large-scale systems.

SimGrid & SMPI: MPI Simulation Framework

SimGrid

- Open-source framework for **simulation of distributed and HPC systems**
- Models:
    - Network characteristics (latency, bandwidth, topology)
    - Host computation speed
- Enables **performance prediction without requiring large physical clusters**

SMPI (Simulated MPI)

- MPI simulation module of SimGrid
- Allows execution of **unmodified MPI applications** in a virtual environment
- Each MPI rank runs as a **simulated process**

Degomme, Augustin, et al. "Simulating MPI applications: the SMPI approach." *IEEE Transactions on Parallel and Distributed Systems* 28.8 (2017): 2387-2400.
https://simgrid.org/doc/latest/Tutorial_MPI_Applications.html
https://simgrid.org/doc/latest/app_smpi.html

Details of SMPI

Collective Implementations

- SMPI integrates **real MPI collective algorithms** from multiple MPI libraries:
- Includes **algorithm selection logic** used in real MPI runtimes.
- Provides **120+ collective algorithm implementations**.
- Runtime automatically chooses algorithm based on: Message size, Number of processes and topology.
- Users can **force a specific algorithm** to study performance impact.

Mini project

1. Install

- Install **SimGrid** (includes **SMPI**) Provides tools to simulate MPI applications

2. Compile MPI Program for Simulation - Compile the application using **SMPI compiler —-** Creates a binary that runs inside the simulator

3. Characterize the Real Cluster

- Measure network **latency** and **bandwidth**
- Use **OSU micro-benchmarks**

4. Create Platform (Network) Model

  - Host compute speed
  - Network latency
  - Network bandwidth

5. Prepare Hostfile

  - Number of nodes
  - Number of processes (cores) per node

6. Run Simulation
Binary + Platform file + Hostfile → Execute with SMPI

Thank You